

AI Transcript: ChatGPT work[5.6 Sol High]

Project Context

I was developing a COMS3011A Lab 1 Todo application using Next.js, SQLite, REST APIs, and React. The AI was used for planning, debugging, reviewing, testing, and documentation assistance. All decisions were reviewed and accepted by me before implementation.

1. Project Structure and Architecture Planning

User:

I need to create a small Todo application. Do I require an architectural style and design patterns?

AI Response:

A lightweight layered architecture was recommended:

UI Layer

|

REST API Layer

|

Service Layer

|

Repository Layer

|

SQLite Database

The Repository pattern was recommended for database separation and a Service layer was recommended for business logic separation.

Decision:

I accepted this because it provided clean separation without unnecessary complexity.

2. Repository Structure Planning

User:

I want balanced file separation and clean REST API practices.

AI Response:

A structure separating application code, database logic, and documentation was recommended:

app/

database/

data/

lib/

docs/

Decision:

The structure was accepted because responsibilities were clearly separated.

3. Database Implementation

User:

How do I implement the database connection?

Requirements:

- Open data/tasks.db.
- Execute database/schema.sql.
- Create the database automatically.
- Support temporary databases for testing.

AI Response:

Use better-sqlite3, initialise the schema automatically, and support :memory: databases for isolated tests.

Decision:

This approach was implemented.

4. Debugging Database and Service Integration Issue

Problem:

The service layer failed to import.

Error:

SyntaxError:

The requested module './taskRepository.js' does not provide an export named 'createTaskRepository'

Investigation:

taskService.js expected a createTaskRepository factory, but taskRepository.js exported individual functions:

createTask()

findTaskById()

findTasks()

updateTask()

archiveTask()

The earlier dynamic import appeared successful because a missing property returned undefined. Static imports correctly failed.

Correction:

The service was updated to use the existing repository exports instead of introducing an unnecessary factory.

Verification:

npm run build passed successfully and integration tests were performed using an in-memory database.

5. REST API Implementation

User:

How do I implement the REST API but follow clean REST practices.

AI Response:

The API was separated into resource endpoints:

GET /api/tasks

POST /api/tasks

GET /api/tasks/[id]

PATCH /api/tasks/[id]

The AI recommended keeping validation and business rules outside the route handlers.

Decision:

The REST structure was implemented.

Verification:

Successful requests included:

POST /api/tasks 201

GET /api/tasks 200

PATCH /api/tasks/[id] 200

GET missing task 404

6. UI Integration

User:

The UI was generated separately using Figma Make. Integrate it into the existing application structure without changing the backend.

AI Response:

The UI should use React components, communicate with the existing REST API, and avoid mock data.

Decision:

The UI was integrated while preserving backend, database, and API logic.

7. Testing Strategy

User:

How do I know that these tests are correct and not hardcoded?

AI Response:

Fixed values are acceptable when they represent requirements. Tests should verify behaviour, not implementation details.

Examples tested:

- Past incomplete tasks become overdue.
- Completed tasks are not overdue.
- Future tasks are not overdue.
- Archived tasks are preserved instead of deleted.

Mutation testing was recommended:

- Introduce a deliberate bug.
- Confirm tests fail.
- Restore the implementation.
- Confirm tests pass.

Decision:

The tests were accepted because they verified required behaviour.

8. Testing Implementation

Implemented tests included:

Unit tests:

- Validation rules.
- Date validation.
- Overdue calculation.

Integration tests:

- Database persistence.
- Creating tasks.
- Updating tasks.
- Sorting.
- Archiving.

Service tests:

- Validation errors.
- Missing tasks.
- Derived overdue information.

Tests used an in-memory SQLite database.

9. Documentation and AI Compliance

AI was used for:

- Architecture planning.
- Debugging.
- Code review.
- Test planning.
- Documentation assistance.

The developer remained responsible for:

- Accepting or rejecting suggestions.
- Verifying implementation.
- Testing functionality.

Summary:

The transcript demonstrates constraints provided before implementation, debugging decisions, correction of unsuitable assumptions, and decisions traceable to the final implementation.